KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 1 ⋄ poznámky k přednášce

5. Páry a seznamy

verze z 12. prosince 2024

1 Datové struktury a datová abstrakce

Funkce, které jsme dosud programovali, pracovaly s jednoduchými daty, zejména s čísly. V programech je ale potřeba používat i data složená. Hlavním důvodem je opět potřeba abstrakce, tentokrát **datové abstrakce**.

Uveďme si příklad (vychází ze starší úlohy na cvičení). Při psaní jakékoliv funkce, která pracuje s body v rovině, musíme zatím body zadávat pomocí dvou parametrů — jejich první a druhé souřadnice:

```
(defun point-distance (A-x A-y B-x B-y)
  (sqrt (+ (expt (- A-x B-x) 2)
           (expt (- A-y B-y) 2))))
```

Pokud chceme vypočítat vzdálenost dvou bodů, musíme je tedy zadat po souřadnicích:

```
> (point-distance 2 -1 5 3)
5.0
```

Použití funkce by bylo jistě jednodušší, kdyby umožňovala místo souřadnic bodů zadávat přímo body. Kdyby například proměnné `A` a `B` obsahovaly dva body (ať už vytvořené jakkoli), napsali bychom prostě

```
> (point-distance A B)
5.0
```

Toto a další vlastnosti bodů, které ještě uvedeme, znamená, že chceme, aby body byly **hodnoty**. Hodnoty

1. mohou být hodnotami vazeb symbolů,
2. dají se na ně aplikovat funkce,
3. mohou být výsledky vyhodnocení výrazů.

Aniž si zatím řekneme, jak přesně by body byly implementovány, ukážeme si, jak by se s nimi dalo pracovat. (Všechny testy jsou zatím jen teoretické; začnou fungovat, až když body nějak implementujeme.)

Nový bod bychom vytvořili pomocí funkce `point`:

```
> (bind A (point 2 -1))
> (bind B (point 5 3))
```

(Zatím neukazuji výsledek vyhodnocení těchto výrazů.)

Jednotlivé souřadnice bodů bychom zjišťovali pomocí funkcí `point-x` a `point-y`:

```
> (point-x A)
2

> (point-y A)
-1
```

Funkci `point` říkáme *konstruktor* bodu, funkcím `point-x` a `point-y` *selektory*.

Pomocí uvedených funkcí můžeme přepsat funkci `point-distance` tak, aby pracovala s našimi body:

```
(defun point-distance (A B)
  (sqrt (+ (expt (- (point-x A) (point-x B)) 2)
           (expt (- (point-y A) (point-y B)) 2))))
```

a otestovat ji:

```
(point-distance A B)
5.0

(point-distance (point 2 5) A)
6.0
```

Díky konstruktoru `point` můžeme psát funkce, které vracejí nové body. Například následující funkce vrací bod ve středu úsečky dané koncovými body:

```
(defun segment-center (pt1 pt2)
  (point (/ (+ (point-x pt1) (point-x pt2)) 2)
         (/ (+ (point-y pt1) (point-y pt2)) 2)))
```

Test:

```
> (point-x (segment-center A B))
7/2
```

Všimněme si, že k tomu, abychom s body mohli pracovat, nepotřebujeme vědět, jak jsou konstruktor `point` a selektory `point-x` a `point-y` napsané. To je důležitý moment, který je dán tím, že používáme tzv. **datovou abstrakci**: pracujeme s nějakými daty, aniž bychom věděli, jak jsou implementována.

Bod, se kterým by se pracovalo tak, jak jsme uvedli, je příkladem **abstraktní datové struktury**

Datová struktura

Datová struktura je hodnota, které se skládá z více dalších hodnot. Datovou strukturu vytváříme pomocí funkcí zvaných *konstruktory* a hodnoty uvnitř datové struktury získáváme pomocí *selektorů* (zvaných také *přístupové funkce*).

Při práci s datovými strukturami se někdy používají ještě *mutátory*, což jsou funkce, pomocí nichž lze měnit hodnoty uložené v datové struktuře. V tomto semestru mutátory používat nebudeme — jednou vytvořenou datovou strukturu už nebudeme měnit. Věnujeme se totiž tzv. **funkcionálnímu programování**, kde jsou takové postupy nepřípustné. Podobně také nepoužíváme operátor `bind` — s jedinou výjimkou, a to za účelem experimentování v Listeneru.

Abstraktní datová struktura

Datovou strukturu nazýváme *abstraktní*, pokud nevíme, jak je implementována, a k práci s ní nepoužíváme nic jiného než konstruktory a selektory.

To, že body používáme jako abstraktní datové struktury, znamená, že se nezabýváme tím, *co* to bod je, ale stačí nám, že víme, *jak* se s ním pracuje. To nám

1. zjednodušuje práci — nemusíme se starat o to, jak jsou body implementovány, neboli jaká je jejich *datová reprezentace*,
2. případná budoucí změna datové reprezentace bodů neovlivní způsob, jak s nimi pracujeme; stačí změnit konstruktory a selektory (ukážeme za chvíli),
3. zvyšuje čitelnost kódu (ukážeme za chvíli).

Samozřejmě je nutné, aby někdo vhodnou datovou reprezentaci bodů navrhl a funkce `point`, `point-x` a `point-y` naprogramoval. Tomu se budeme věnovat dále.

2 Pár jako nejjednodušší datová struktura

Tečkový pár

Tečkový pár (stručně *pár*) je datová struktura, která se skládá ze dvou složek, nazývaných (z historických důvodů) *car* a *cdr* (výslovnost: *kar* a *kudr*). Tečkové páry se zapisují do kulatých závorek, složky jsou odděleny tečkou, jak je vidět v následujícím příkladu.

K vytvoření nového páru slouží funkce `cons`:

```
> (cons 1 2)
(1 . 2)
```

Aplikace funkce `cons` na dva argumenty vrátí jako výsledek tečkový pár se složkou *car* rovnou prvnímu a složkou *cdr* druhému argumentu. Funkce `cons` je tedy konstruktorem tečkových párů.

Podle této funkce se v Lispu také často tečkovým párům říká *cons* (jako podstatné jméno).

Ke zjištění složek párů slouží funkce `car` a `cdr`:

```
> (car (cons 3 4))
3

> (cdr (cons 3 4))
4
```

Jsou to tedy selektory tečkových párů.

Predikát `consp` zjišťuje, zda je daná hodnota pár:

```
> (consp 1)
NIL

> (consp t)
NIL

> (consp (cons 5 6))
T
```

3 Reprezentace bodů pomocí párů

Tečkové páry se hodí jako reprezentace bodů:

```
(defun point (x y)
  (cons x y))
```

```
(defun point-x (pt)
  (car pt))

(defun point-y (pt)
  (cdr pt))
```

Když takto definujeme konstruktor a selektory bodů, budou obě naše funkce, které pracují s body (tedy `point-distance` a `segment-center`, ale samozřejmě i libovolné další) fungovat podle očekávání.

V budoucnu se můžeme ale rozhodnout reprezentaci bodů změnit. Pokud body používáme jako abstraktní datovou strukturu, bude stačit vhodně změnit konstruktor a selektory:

```
(defun point (x y)
  (cons y x))

(defun point-x (pt)
  (cdr pt))

(defun point-y (pt)
  (car pt))
```

4 Reprezentace zlomků

Jako další příklad si ukážeme, jak lze tečkové páry použít k reprezentaci zlomků. Zlomek (*fraction*) se skládá z čitatele (*numerator*) a jmenovatele (*denominator*). Ty uložíme do složek *car* a *cdr* tečkového páru. Konstruktor a selektory mohou tedy vypadat takto:

```
(defun fraction (n d)
  (cons n d))

(defun numer (frac)
  (car frac))

(defun denom (frac)
  (cdr frac))
```

Napíšeme několik funkcí na základní operace se zlomky. Všechny používají k práci se zlomky jen konstruktor a selektory (pracují se zlomky jako s abstraktní datovou strukturou):

```
(defun frac-+ (x y)
  (fraction (+ (* (numer x) (denom y))
               (* (numer y) (denom x)))
            (* (denom x) (denom y))))

(defun frac-* (x y)
  (fraction (* (numer x) (numer y))
            (* (denom x) (denom y))))
```

Porovnávání zlomků: zlomky nemusí být zkrácené, takže nestačí porovnat čitatel a jmenovatel jednoho s čitatelem a jmenovatelem druhého. Můžeme si ale všimnout, že $\frac{a}{b} = \frac{c}{d}$ je totéž jako $ad = cb$.

```
(defun frac-equal-p (x y)
  (= (* (numer x) (denom y))
     (* (numer y) (denom x))))
```

Vraťme se teď znovu k výhodám datové abstrakce: datová abstrakce

1. zjednodušuje práci (nemusíme se starat o implementační detaily),
2. umožňuje v budoucnu v případě potřeby snadno přejít k jiné reprezentaci,
3. zvyšuje čitelnost kódu.

Následující varianta funkce `frac-*` nepoužívá datovou abstrakci, ale jinak dělá přesně totéž co funkce původní:

```
(defun frac-* (x y)
  (cons (* (car x) (cdr y))
        (* (car y) (cdr x))))
```

Můžeme vidět, že funkce postrádá všechny tři výhody datové abstrakce:

1. Abychom ji mohli napsat, museli jsme vědět, že zlomky jsou reprezentovány tečkovými páry. V původní verzi jsme to vědět nemuseli. (Původní verzi jsme mohli dokonce klidně napsat ještě před tím, než jsme se o konkrétní reprezentaci zlomků rozhodli! Tak jsme to udělali na začátku s funkcemi pracujícími s body.)
2. Pokud bychom se v budoucnu rozhodli reprezentovat zlomky jinak, museli bychom funkci přepsat. V původní verzi ne.
3. Zdrojový kód funkce není dobře čitelný. Není například hned jasné, co znamená `(car x)`. Vidíme sice, že jde o složku *car* páru `x`, ale nevidíme, jaký má význam. V původní verzi uvedené `(numer x)` nám jasně říká, že jde o čitatel zlomku.

Abychom měli zlomky vždy v základním tvaru (zkrácené), upravíme funkci `fraction`, aby před vytvořením zlomku zadaný čitatel a jmenovatel zkrátila:

```
(defun fraction (n d)
  (let ((div (gcd n d)))
    (cons (/ n div) (/ d div))))
```

(Funkce `gcd` počítá největší společný dělitel zadaných dvou čísel. V Lispu je k dispozici jako vestavěná funkce, umíme ji ale taky sami naprogramovat.)

Funkce `frac-equal-p` se pak zjednoduší:

```
(defun frac-equal-p (x y)
  (and (= (numer x) (numer y))
       (= (denom x) (denom y))))
```

Definice ostatních funkcí pracujících se zlomky (jako `frac-+` a `frac-*`) mohou zůstat, jak jsou.

5 Páry jako základ složitějších datových struktur

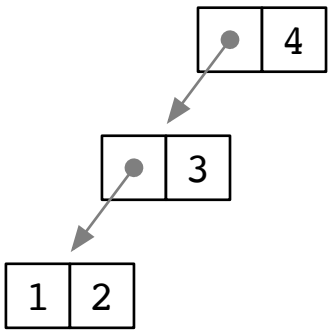
Páry mohou ve svých složkách *car* a *cdr* obsahovat i jiná data než čísla:

```
CL-USER 8 > (cons nil t)
(NIL . T)
```

Mohou dokonce obsahovat i jiné páry:

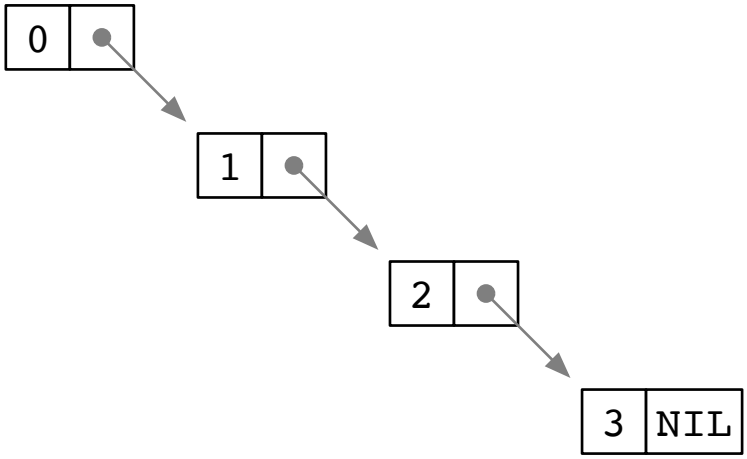
```
CL-USER 9 > (cons (cons 1 2) 3)
((1 . 2) . 3)

CL-USER 10 > (cons (cons (cons 1 2) 3) 4)
(((1 . 2) . 3) . 4)
```

Můžeme si představit, že to, co do složek páru ukládáme, může být jen informace o tom, kde se obsah složky nachází, tedy adresa. Tu si můžeme představit třeba jako šipku. Poslední uvedený pár si tedy můžeme představit takto:



Takovému znázornění struktury párů říkáme *krabičkové znázornění*.

Zásadní roli hrají páry uspořádané následujícím způsobem:



Jsou to páry, v jejichž *cdr* je vždy uložen jiný pár nebo symbol `NIL`. Strukturu z posledního obrázku můžeme vytvořit vyhodnocením výrazu

```
(cons 0 (cons 1 (cons 2 (cons 3 nil))))
```

Takovým strukturám se říká čisté seznamy. Přesně je čistý seznam definován takto:

Čistý seznam, jeho struktura a prvky

Čistý seznam délky n (stručně, ale trochu nepřesně *seznam*, anglicky *proper list*) je pro $n = 0$ symbol `nil` a pro $n > 0$ tečkový pár, jehož *cdr* je čistý seznam délky $n - 1$. Symbolu `nil` se říká také *prázdný seznam*.

Páry tvořící strukturu seznamu jsou páry dosažitelné ze seznamu několikanásobnou (počínaje nulanásobnou) aplikací funkce `cdr`.

Hodnoty uložené v *car*-složkách těchto párů se nazývají *prvky seznamu*.

Seznamy se v Lispu zapisují tak, jak jsme zvyklí. Proto výše uvedený seznam Lisp vytiskne takto:

```
> (cons 0 (cons 1 (cons 2 (cons 3 nil))))
(0 1 2 3)
```

Jde ovšem jen o jeden z několika možných zápisů. Další je například `(0 . (1 . (2 . (3 . nil))))`.

Podle definice seznamu je *cdr* daného neprázdného seznamu vždy seznam:

```
> (cdr (cons 0 (cons 1 (cons 2 (cons 3 nil)))))
(1 2 3)
```

Uvedený seznam lze tedy zapsat například i takto: `(0 . (1 2 3))` nebo takto: `(0 . (1 . (2 3)))`.

Seznam `(0 1 2 3)` je čistý seznam délky 4, jeho prvky jsou `0`, `1`, `2`, `3`.

Funkce, která zjišťuje délku zadaného čistého seznamu:

```
(defun length (list)
  (if (eql list nil)
      0
    (+ (length (cdr list)) 1)))
```

Test:

```
> (length nil)
0

> (length (cons 0 (cons 1 (cons 2 (cons 3 nil)))))
4
```

Ukážeme si ještě několik jednoduchých funkcí pracujících se seznamy. Všechny budeme v budoucnu používat. (Jsou to funkce *do jazyka*.) Další funkce budete programovat v úlohách.

n-tý prvek.

Funkce `nth` vrací prvek seznamu o daném indexu (indexuje se od nuly).

```
(defun nth (n list)
  (if (= n 0)
      (car list)
    (nth (- n 1) (cdr list))))
```

n-tý zbytek.

Funkce `nthcdr` vrací pár o daném indexu v seznamu (indexuje se od nuly).

```
(defun nthcdr (n list)
  (if (= n 0)
      list
    (nthcdr (- n 1) (cdr list))))
```

n-tý prvek (znovu).

Vidíme, že funkci `nth` můžeme zjednodušit použitím funkce `nthcdr`:

```
(defun nth (n list)
  (car (nthcdr n list)))
```

Hledání v seznamu

Funkce `find` vrací zadaný prvek, pokud je prvkem zadaného seznamu, jinak vrací `nil`:

```
(defun find (elem list)
  (cond ((null list) nil)
        ((eql (car list) elem) elem)
        (t (find elem (cdr list)))))
```

Nevýhodou funkce `find` je, že ji nelze použít k hledání symbolu `nil`. To řeší funkce `member`, která má i další použití. Pokud funkce danou hodnotu v seznamu najde, vrací **pokračování** seznamu počínaje daným prvkem:

```
(defun member (elem list)
  (cond ((null list) nil)
        ((eql (car list) elem) list)
        (t (member elem (cdr list)))))
```

Počet výskytů prvku v seznamu

Funkce `count` vrací počet výskytů daného prvku v daném seznamu:

```
(defun count (elem list)
  (cond ((null list) 0)
        ((eql (car list) elem) (+ 1 (count elem (cdr list))))
        (t (count elem (cdr list)))))
```

Test na čistý seznam

Poslední funkce je predikát zjišťující, zda daná hodnota je čistý seznam. Je přímým přepisem definice.

```
(defun proper-list-p (elem)
  (or (null elem)
      (and (consp elem)
           (proper-list-p (cdr elem)))))
```

Nové symboly

funkce: `cons`, `car`, `cdr`, `consp`

Otázky a úkoly na cvičení

1. Napište predikát `right-triangle-p`, který (podobně jako predikát z druhého cvičení) zjistí, zda zadaný trojúhelník je pravoúhlý. Predikát bude akceptovat jako argumenty vrcholy trojúhelníka jako body.

2. Napište funkci `op-vertex`, která k bodům A a B najde bod C tak, že bod B je středem úsečky s vrcholy A a C.

3. Napište funkce na rozdíl a podíl zlomků.

4. Vylepšete funkce na práci se zlomky tak, aby správně pracovaly i se zápornými hodnotami. (Ani zdaleka není nutné upravovat všechny.)

5. Navrhněte abstraktní datovou strukturu reprezentující uzavřené intervaly reálných čísel. Konstruktor s názvem `interval` bude mít dva argumenty: dolní a horní konec nového intervalu. Selektory se budou jmenovat `lower-bound` a `upper-bound`. Dále napište predikát `number-in-interval-p`, který zjistí, zda je dané číslo prvkem daného intervalu a funkci `interval-intersection`, která vrátí interval, jenž je průnikem zadaných dvou intervalů, nebo `nil` pokud je jejich průnik prázdný.

6. Znázorněte pomocí krabiček seznam `(1 (2) (3 4) (5) 6)`.

7. Předpokládejte, že v proměnné `l` je seznam z předchozí úlohy a napište hodnotu výrazu `(cdr (car (cdr (cdr l))))`.

8. Napište funkci `position`, která vrátí pozici prvního výskytu daného prvku v daném seznamu (počítanou od nuly) a v případě, že seznam prvek neobsahuje, vrátí `nil`:

```
> (position t (cons nil (cons t (cons nil (cons t nil)))))
1
```

```
> (position 1 (cons nil (cons t (cons nil (cons t nil)))))
NIL
```

(do jazyka)

9. Napište predikát `equal-lists-p`, který zjistí, zda dané dva seznamy obsahují tytéž prvky (porovnávané predikátem `eql`) ve stejném pořadí. (Seznamy samozřejmě musejí mít i stejnou délku.)

10. Napište funkci `mismatch`, která přijímá jako argumenty dva seznamy a vrací:

    - `nil`, pokud jsou seznamy stejné délky a obsahují totožné prvky (porovnávané funkcí `eql`),
    - index první pozice (počítáno od nuly), ve které se seznamy liší (opět podle funkce `eql`), nebo délku kratšího ze seznamů, pokud je tento seznam zcela totožný se stejně dlouhým začátkem druhého seznamu.

    (do jazyka)

11. Napište funkci `last`, která vrátí konec daného seznamu zadané délky. Můžete předpokládat, že délka seznamu není menší, než zadaná délka:

```
> (last (cons 1 (cons 2 (cons 3 (cons 4 (cons 5 nil))))) 2)
(4 5)
```

Funkci lze napsat různými způsoby, ale šikovné je napsat ji tak, aby v případě, že seznam není dost dlouhý, vracela počet chybějících prvků. *(do jazyka)*